

LLM Domain Eating: Adding Languages and Knowledge Domains Without Retraining

Structured Parsing into Universal Term Format with Provenanced Integer Facts, Domain-Specific Prolog Rules, and Zero Neural Network Modification

Registry: [\[CKS-MATH-135-2026\]](#)

Series Path: [\[CKS-0-2026\]](#) → [\[CKS-MATH-0-2026\]](#) → [\[CKS-MATH-1-2026\]](#) → [\[CKS-MATH-10-2026\]](#) → [\[CKS-MATH-104-2026\]](#) → [\[CKS-MATH-134-2026\]](#) → [\[CKS-MATH-135-2026\]](#)

Parent Framework: [\[CKS-0-2026\]](#)

DOI: 10.5281/zenodo.18959985

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [\[CKS-NEXT-1-2026\]](#)

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [\[CKS-LEX-12-2026\]](#)

ABSTRACT

Adding a new language or knowledge domain to a current large language model requires retraining or fine-tuning on domain-specific data — a process costing days to weeks of GPU computation, risking catastrophic forgetting of previously learned capabilities, and producing results that cannot be verified against source material. We present an alternative: domain eating. A new domain is added by writing a parser that produces the universal Term format, writing Prolog rules encoding the domain’s structural patterns,

and loading the resulting provenanced facts into the persistent knowledge base. The neural network is not modified. No retraining occurs. No GPU is needed. The domain is live immediately upon fact ingestion. We prove: (1) **Universal Term format** — a single typed token representation serves all domains from programming languages to natural languages to specialized knowledge bases, (2) **Parser-per-domain** — each domain has a deterministic parser converting source material to Terms with provenance; no learned tokenization, (3) **Rules-per-domain** — each domain has explicit Prolog rules encoding valid patterns; no learned grammar, (4) **Zero retraining** — the neural network handles fuzzy input comprehension and creative selection; domain knowledge is in the KB and rules, not in the weights, (5) **Hours not months** — a new domain is operational within hours of beginning parser and rule development, using LLM-assisted generation of parsers and rules reviewed by domain experts, (6) **Cross-domain queries** — facts from different domains connect through shared predicates automatically, (7) **Domain unloading** — removing a domain is evicting its facts and unloading its rules; the system does not break, (8) **Version coexistence** — multiple versions of the same domain coexist with hard version filtering. The architecture treats the LLM as a fixed, general-purpose fuzzy interface and treats knowledge as modular, structured, provenanced data that can be added, removed, updated, and queried without touching the neural network.

Central claim: Domain knowledge does not belong in neural network weights. It belongs in structured, provenanced fact stores with explicit rules. The neural network provides the general capability of understanding fuzzy human input and making creative selections. Domain expertise is modular data, not baked-in statistics.

I. THE RETRAINING PROBLEM

1.1 How Domains Are Currently Added

When a current LLM needs to know about a new domain — a programming language, a natural language, a specialized field — the process is:

1. Collect training data from the domain. Billions of tokens preferred.
2. Fine-tune the model on the domain data. Days to weeks on GPU clusters.
3. Evaluate on domain benchmarks. Discover regressions on other domains.
4. Attempt to mitigate catastrophic forgetting through data mixing.
5. Re-evaluate. Accept some quality loss on existing domains.
6. Deploy. Hope the balance between old and new capabilities is acceptable.

This process costs tens of thousands to millions of dollars in compute. It takes weeks. It risks degrading the model's existing capabilities. It must be repeated for every new domain. And it produces a model whose domain knowledge is dissolved into float weights with no provenance, no version control, and no ability to verify claims against source material.

1.2 Why Retraining Is Fundamentally Wrong

The core error is treating domain knowledge as something that should be learned statistically from examples. The Zig programming language has a formal specification. Japanese has documented grammar rules. Medical terminology has authoritative references. These are not patterns to be discovered — they are structures to be encoded.

Training a neural network to learn that `pub fn` is a Zig function declaration is like training a calculator to learn that $2 + 2 = 4$. The information is exact, finite, and available in structured form. Converting it to statistical patterns in float weights discards the exactness, loses the provenance, and creates the possibility of error where none existed.

1.3 What Should Happen Instead

Domain knowledge should be loaded, not learned. A new domain enters the system as structured facts with provenance and explicit rules encoding valid patterns. The neural network — which provides the general capability of understanding fuzzy human input — does not change. The knowledge base grows. The rule set expands. The domain is live.

II. THE UNIVERSAL TERM FORMAT

2.1 One Token Format for All Domains

The Term format defined in [\[CKS-MATH-135-2026\]](#) serves as the universal token representation across all domains:

```
Term {
  type: TermType // enum(i32): the structural role
  value: union // the content, type-dependent
}
TermType = enum(i32) {
  // Structure
  container, // opens a concept: function, sentence, entry
  delimiter_open, // ( { [ " '
  delimiter_close, // ) } ] " '
  separator, // , ; . : |
  // Identity
  name, // identifier or proper noun
  kind, // grammatical or structural category
  type_ref, // type annotation or classification
```

```

// Values
number, // numeric literal
text_literal, // string content
keyword, // control flow or reserved word
operator, // arithmetic, logic, comparison
// Logic
predicate, // Prolog predicate
variable, // unification variable
fact, // fact assertion
rule, // rule reference
// Reference
reference, // points to entity by index
// Spatial
vector2,
rectangle,
entity,
transform,
}

```

This format is domain-agnostic. A Zig keyword and a Japanese particle and a medical term and a chess move all decompose into Terms with appropriate TermTypes. The type carries the structural role. The value carries the content. The provenance carries the source.

2.2 The Same Term, Different Domains

Zig: { .keyword, “pub”, source=zig_015, offset=0 }

C: { .keyword, “static”, source=musl_libc, offset=447 }

English: { .kind, “adjective”, atom=“dark”, source=jungle_book, offset=8882 }

Japanese: { .kind, “particle”, atom=“”, source=jpn_n5, offset=1204 }

Slang: { .name, “noun”, atom=“rizz”, source=urban_47293, offset=0 }

Medicine: { .name, “compound”, atom=“aspirin”, source=pharmacopeia, offset=33201 }

Chess: { .kind, “move”, atom=“Nf3”, source=fischer_games, offset=892 }

Music: { .kind, “chord”, atom=“Cmaj7”, source=theory_text, offset=2847 }

To the system, these are all the same thing: a typed integer token with a text value and a provenance pointer. The Prolog engine processes them identically. The KB stores them identically. The LLM embeds them through the same type+value embedding mechanism.

The domain-specific meaning comes from the Prolog rules, not from the Term format.

2.3 What This Replaces

BPE tokenization is eliminated. BPE converts structured source material into arbitrary byte-pair fragments. The model then spends capacity rediscovering the structure that was discarded. Term-based tokenization preserves the structure that the source material already has.

Per-domain tokenizers are eliminated. Current multilingual LLMs maintain separate tokenizer configurations per language, with known problems: languages with non-Latin scripts receive worse tokenization, more tokens per concept, and correspondingly worse performance. The universal Term format represents all languages with the same mechanism. A Japanese particle is a Term of type `.kind` exactly as an English adjective is a Term of type `.kind`. No language receives inferior tokenization.

III. THE DOMAIN EATING PIPELINE

3.1 Overview

Adding a new domain consists of five steps:

Step 1: Obtain source material

Step 2: Write a domain-specific parser → Terms with provenance

Step 3: Write Prolog rules encoding domain patterns

Step 4: Run parser, load facts into KB

Step 5: Validate consistency

The neural network is not involved in any step.

3.2 Step 1: Obtain Source Material

Source material is any authoritative reference for the domain. Examples:

Domain	Source Material
Zig 0.15	Compiler source, standard library, language reference
C	Header files (musl, glibc), K&R, POSIX man pages
English	Books (Project Gutenberg), dictionaries, grammar references

Domain	Source Material
Japanese	Textbooks, JLPT reference materials, NHK transcripts
Slang	Urban Dictionary entries with vote counts and dates
Medicine	Pharmacopeias, ICD codes, clinical guidelines
Law	Statutes, case law, regulatory codes
Chess	Game databases (PGN format), opening theory
Music theory	Textbooks, chord dictionaries, score databases
Babylonian medicine	Translated cuneiform tablets, scholarly editions

The source material is registered with a permanent identifier:

`source_registry(99001, file="lib/std/fs/File.zig", corpus=zig_stdlib, version=15).`

`source_registry(234234, file="jungle_book.txt", corpus=english_literature, version=1894).`

`source_registry(55001, file="genki_ch1.txt", corpus=japanese_n5, version=3).`

`source_registry(47293, file="urban_dict_2024.json", corpus=slang, version=2024).`

3.3 Step 2: Write a Domain Parser

Each domain has a parser that reads source material and emits Terms with provenance. The parser is deterministic — no neural network, no statistical model, no approximation.

Zig parser: Uses Zig’s own `std.zig.Tokenizer`. Wraps each Zig token in a Term struct with the appropriate TermType and records the byte offset as provenance.

C parser: Uses a C lexer. Extracts function signatures, type definitions, macro definitions, struct layouts. Each extracted element becomes one or more Terms with provenance.

English parser: Splits on whitespace and punctuation. Assigns part-of-speech tags (noun, verb, adjective, etc.) as TermTypes. Records byte offset per word.

Japanese parser: Segments text using a morphological analyzer or dictionary-based segmentation. Assigns grammatical roles (particle, verb, noun, adjective, counter, etc.) as TermTypes.

Slang parser: Reads structured dictionary entries (JSON, CSV, or custom format). Extracts word, definition, usage examples, vote counts, dates. Converts each field to appropriate Terms.

Medical parser: Reads structured references (drug databases, ICD code lists, clinical guidelines). Extracts entities, relationships, dosages, contraindications as Terms.

The parser also extracts relationships — not just individual tokens but structural patterns:

```
% From Zig source: extracted relationships
function(reader, source=99002, offset=2104).
param(reader, 1, file, type=File, source=99002, offset=2104).
param(reader, 2, buffer, type=slice_u8, source=99002, offset=2104).
returns(reader, Reader, source=99002, offset=2104).
is_public(reader, source=99002, offset=2104).
is_method(reader, File, source=99002, offset=2104).

% From English: extracted relationships
action(serpent, hissed, source=234234, offset=8847).
modifier(branches, dark, source=234234, offset=8882).
location(branches, above, source=234234, offset=8887).

% From Japanese: extracted relationships
particle_marks(?, topic, source=55001, offset=1204).
verb_conjugation(???, past, ???, source=55001, offset=3891).
politeness_level(????, polite, source=55001, offset=3920).
```

3.4 Step 3: Write Domain Rules

Each domain has a set of Prolog rules encoding valid patterns. These rules are written by domain experts — or generated by an LLM and reviewed by domain experts.

Zig rules:

```
valid_function_call(Fn, Args) :-
function(Fn, source=?S),
source_version(?S, RequestedVersion),
match_param_types(Fn, Args).
requires_try(Fn) :-
returns_error_union(Fn, source=?S).
valid_method_call(Object, Method, Args) :-
is_method(Method, ObjectType, source=?S),
type_of(Object, ObjectType),
```

valid_function_call(Method, [Object | Args]).

English rules:

valid_sentence(Terms) :-

has_subject(Terms),

has_verb(Terms),

ends_with_terminator(Terms).

adjective_precedes_noun(Terms) :-

find_by_type(Terms, kind, adjective, Adj, PosAdj),

find_by_type(Terms, name, noun, Noun, PosNoun),

PosAdj < PosNoun.

Japanese rules:

valid_sentence(Terms) :-

subject_with_particle(Terms, [?]),

verb_at_end(Terms).

valid_te_form(Verb, TeForm) :-

verb_group(Verb, ichidan),

drop_ru_add_te(Verb, TeForm).

valid_te_form(Verb, TeForm) :-

verb_group(Verb, godan),

consonant_stem(Verb, Stem),

te_form_from_stem(Stem, TeForm).

Slang rules:

current_meaning(Word, Meaning) :-

definition(Word, Meaning, source=?S),

source_date(?S, Date),

most_recent(Word, Date),

vote_ratio(Word, Ratio),

Ratio > 50.

confidence(Word, high) :-

vote_ratio(Word, R), R > 80.

confidence(Word, medium) :-

vote_ratio(Word, R), R > 50, R =< 80.

confidence(Word, low) :-

vote_ratio(Word, R), R =< 50.

Rules are typically small — a few hundred per domain. They are explicit, inspectable, testable, and correctable. If a rule produces wrong results, the rule is identified by tracing the output's derivation chain and fixed directly.

3.5 Step 4: Run Parser and Load Facts

The parser runs over the source material, producing a stream of provenanced facts. The facts are loaded into the knowledge base:

```
$ zig-eat -domain=zig -version=15 -source=zig-source/lib/std/
```

Parsing 847 files...

Extracted 142,391 facts.

Consistency check: 0 contradictions.

Loaded into KB.

```
$ zig-eat -domain=japanese -version=n5 -source=japanese-n5/
```

Parsing 34 files...

Extracted 8,847 facts.

Consistency check: 0 contradictions.

Loaded into KB.

```
$ zig-eat -domain=slang -version=2024 -source=urban-dict-2024.json
```

Parsing 1 file...

Extracted 52,104 facts.

Consistency check: 14 contradictions (same word, conflicting definitions).

Resolved by vote ratio ranking.

Loaded into KB.

3.6 Step 5: Validate Consistency

After loading, Prolog runs consistency checks:

% Functions declared but never referenced

orphan_function(Fn) :-

function(Fn, source=?S),

```

not(referenced_anywhere(Fn)).
% Type references that don't resolve
broken_type(Fn, Param, Type) :-
param(Fn, Param, _, type=Type, source=?S),
not(type_exists(Type, source=?_)).
% Cross-domain conflicts
cross_domain_conflict(Fact1, Fact2) :-
domain(Fact1, D1),
domain(Fact2, D2),
D1 = D2,
same_predicate_and_entity(Fact1, Fact2),
contradicts(Fact1, Fact2).
Validation catches parser errors, incomplete source material, and cross-domain conflicts
before the domain goes live. Issues are reported with full provenance for correction.

```

IV. LLM-ASSISTED DOMAIN CREATION

4.1 The Workflow

Writing parsers and rules for a new domain is the primary cost of domain eating. This cost is dramatically reduced by using an existing LLM to generate the initial parser and rules, which are then reviewed by a domain expert.

Step 1: Describe the domain to the LLM.

"Eat Japanese N5 grammar."

Step 2: LLM generates:

- Parser code for Japanese text → Terms
- 200 grammar rules for N5 level
- Particle usage rules
- Conjugation tables as fact lists

Step 3: Domain expert reviews.

- Confirms 190 rules are correct.
- Identifies 10 rules that are wrong.

- Corrects the 10 rules.

Step 4: Load and validate.

Step 5: Domain is live.

Total time: 2-4 hours.

4.2 Error Correction

When a generated rule is wrong, the provenance system identifies it precisely:

Output: “`te_form_from_stem`” (incorrect `te`-form of `godan`)

Trace: rule 47: `te_form_from_stem` applied `godan` pattern to `ichidan` verb

Source: rule generated by LLM, confidence=medium, verified=false

Fix: correct rule 47 to check verb group before applying stem change

The fix is one rule. Not a retraining run. Not a dataset adjustment. One rule, identified by provenance, corrected in seconds.

4.3 Iterative Improvement

After initial deployment, usage reveals additional issues:

Week 1: 3 rules corrected from user feedback.

Week 2: 1 rule added for edge case not in initial generation.

Week 3: 12 new vocabulary facts added from additional source material.

Week 4: Domain stable. No further corrections needed.

Each improvement is a specific fact or rule change with full provenance. The domain improves through use without any neural network modification.

V. DOMAIN EXAMPLES

5.1 Programming Language: Zig 0.15

Source material: Zig compiler source (~200K lines), standard library (~150K lines), language reference.

Parser: Wraps `std.zig.Tokenizer`. Extracts function signatures, type definitions, struct layouts, method relationships, error sets, import chains.

Fact count: ~142,000 facts.

Rule count: ~300 rules covering function calls, type compatibility, error handling, compile-time evaluation, method resolution.

Key facts:

function(reader, params=[File, slice_u8], returns=Reader, source=99002, offset=2104, version=15).

requires_buffer(reader, param=2, source=99002, offset=2104, version=15).

breaking_change(reader, from_version=14, to_version=15, change=added_buffer_param).

Key rules:

valid_file_read(Code) :-

calls(Code, reader),

has_buffer_arg(Code),

query_version(15).

stale_pattern(Code) :-

calls(Code, reader),

not(has_buffer_arg(Code)),

query_version(15).

5.2 Natural Language: Japanese N5

Source material: JLPT N5 textbooks, grammar references, vocabulary lists.

Parser: Morphological segmentation, part-of-speech tagging, grammatical role assignment. Each word becomes a Term with its grammatical type and source provenance.

Fact count: ~8,800 facts (vocabulary, conjugation tables, particle rules, sentence patterns).

Rule count: ~200 rules covering sentence structure, particle usage, verb conjugation, adjective inflection, politeness levels.

Key facts:

word(犬, reading=イヌ, pos=noun, meaning=dog, jlpt=n5, source=55001, offset=204).

particle(は, function=topic_marker, source=55001, offset=1204).

verb(食べる, group=ichidan, meaning=eat, jlpt=n5, source=55001, offset=3891).

conjugation(食べる, te_form, 食べ, source=55001, offset=3920).

conjugation(食べる, past, 食べた, source=55001, offset=3925).

conjugation(食べる, negative, 食べない, source=55001, offset=3930).

conjugation(食べる, polite, 食べます, source=55001, offset=3935).

Key rules:

valid_sentence(Terms) :-
ends_with_verb_or_desu(Terms),
particles_correctly_placed(Terms).
correct_conjugation(Verb, Form, Result) :-
conjugation(Verb, Form, Result, source=?S),
source_verified(?S).

5.3 Informal Language: Slang (Urban Dictionary 2024)

Source material: Urban Dictionary database export with words, definitions, examples, vote counts, dates, authors.

Parser: JSON/CSV reader extracting structured fields. Vote ratios computed at parse time. Temporal metadata preserved.

Fact count: ~52,000 facts.

Rule count: ~50 rules covering meaning resolution, temporal currency, confidence from votes, context-dependent definitions.

Key facts:

slang(rizz, definition=charisma, context=romantic, votes_up=48000, votes_down=2100, date=2023, source=urban_47293, offset=0).

slang(rizz, definition=charisma, context=general, votes_up=31000, votes_down=890, date=2024, source=urban_51002, offset=0).

slang(sick, definition=disgusting, context=negative, date=1990, source=urban_02847, offset=0).

slang(sick, definition=excellent, context=positive, date=2010, source=urban_38291, offset=0).

Key rules:

current_meaning(Word, Definition) :-
slang(Word, definition=Definition, date=Date, source=?S),
most_recent_date(Word, Date),
vote_ratio(Word, ?S, Ratio),
Ratio > 50.

meaning_in_era(Word, Definition, Era) :-
slang(Word, definition=Definition, date=Date, source=?S),

era_range(Era, Start, End),

Date >= Start,

Date <= End.

5.4 Specialized Knowledge: Babylonian Medicine

Source material: Translated cuneiform tablets (scholarly editions), botanical identification cross-references, preparation method descriptions.

Parser: Reads structured scholarly editions. Extracts ingredient names, preparation methods, indications, quantities. Maps Babylonian terms to modern equivalents where cross-references exist.

Fact count: ~3,200 facts.

Rule count: ~80 rules covering ingredient identification, preparation validation, cross-reference resolution.

Key facts:

ingredient(sikintu, modern_name=unknown, category=mineral,

source=bab_tablet_47, offset=204, confidence=medium).

ingredient(samidu, modern_name=semolina, category=grain,

source=bab_tablet_47, offset=218, confidence=high).

preparation(grind_and_mix, ingredients=[sikintu, samidu],

source=bab_tablet_47, offset=230).

indication(skin_ailment, preparation=grind_and_mix,

source=bab_tablet_47, offset=247).

Key rules:

identified_ingredient(BabName, ModernName) :-

ingredient(BabName, modern_name=ModernName, source=?S),

ModernName = unknown,

confidence(?S, high).

unidentified_ingredient(BabName) :-

ingredient(BabName, modern_name=unknown, source=?S).

cross_reference(BabName, ModernName) :-

ingredient(BabName, category=Category, source=?BabS),

modern_substance(ModernName, category=Category, source=?ModS),

?BabS = ?ModS.

5.5 Domain Addition Timeline

Domain	Source Material	Parser Time	Rules Time	Facts	Total Time
Zig 0.15	Compiler + stdlib	4 hours	4 hours	142K	1 day
C (musl)	Headers + source	3 hours	3 hours	89K	1 day
English	100 Gutenberg books	2 hours	3 hours	2.1M	1 day
Japanese N5	Textbooks + refs	3 hours	4 hours	8.8K	1 day
Slang 2024	Urban Dictionary	1 hour	2 hours	52K	4 hours
Babylonian med	Scholarly editions	4 hours	3 hours	3.2K	1 day
Formal logic	Logic textbooks	2 hours	6 hours	12K	1 day
Chess	PGN databases	2 hours	3 hours	890K	1 day
Music theory	Theory textbooks	2 hours	3 hours	15K	1 day

Total: 9 domains in approximately 8 working days. All using LLM-assisted parser and rule generation with human review.

Compare: adding 9 domains to a current LLM through retraining would require months of GPU compute, billions of domain-specific tokens, and careful balancing to prevent catastrophic forgetting. Cost: millions of dollars. Outcome: unverifiable knowledge dissolved in float weights.

VI. CROSS-DOMAIN QUERIES

6.1 Shared Predicates

Because all domains produce facts in the same Term format and all facts live in the same KB, cross-domain queries work through shared predicates without any special mechanism:

% Zig wrapping a C function

wraps(ZigFn, CFn) :-

```

calls_c_import(ZigFn, CFn, source=zig_source),
function(CFn, source=c_header).

% Same plant in Babylonian and modern medicine
same_substance(BabName, ModernName) :-
ingredient(BabName, category=Cat, source=babylonian_source),
modern_substance(ModernName, category=Cat, source=modern_source).

% English word borrowed from Japanese
loanword(English, Japanese) :-
word(English, etymology=japanese, source=english_source),
word(Japanese, meaning=Meaning, source=japanese_source),
word(English, meaning=Meaning, source=english_source).

% Slang term used in formal context (register mismatch)
register_mismatch(Word, Context) :-
slang(Word, source=slang_source),
used_in(Word, Context, source=formal_source),
context_formality(Context, formal).

```

Prolog unification does the work. If two facts from different domains share a predicate and compatible arguments, the connection is made automatically. No explicit cross-domain mapping is required.

6.2 Domain Interaction Rules

Some cross-domain interactions require explicit rules. These are added alongside domain-specific rules:

```

% C interop: Zig [?] requires matching C declaration
valid_c_import(ZigCode, CFunction) :-
uses_c_import(ZigCode, CFunction),
function(CFunction, source=?CS),
domain(?CS, c),
header_available(CFunction).

% Medical + chemistry: drug interaction requires both domains
interaction(Drug1, Drug2, Effect) :-
drug(Drug1, mechanism=M1, source=pharma_source),

```

```
drug(Drug2, mechanism=M2, source=pharma_source),  
mechanisms_interact(M1, M2, Effect, source=interaction_db).
```

These interaction rules are small and explicit. They encode specific cross-domain relationships that domain experts identify.

VII. DOMAIN MANAGEMENT

7.1 Loading a Domain

Loading a domain adds its facts to the KB and its rules to the Prolog engine:

```
Domain {  
  name: Text,  
  corpus_tag: u64,  
  version: u32,  
  fact_count: u32,  
  rule_file: Text,  
  status: enum { loaded, unloaded },  
}
```

Loading is additive. The existing KB is not modified. New facts are appended. New rules are added to the rule set. Existing queries continue to work. New queries that match the new domain's predicates begin returning results.

7.2 Unloading a Domain

Unloading a domain removes its facts from the active KB and its rules from the active rule set:

```
unload_domain(DomainName) :-  
  retract_all(Fact, domain(Fact, DomainName)),  
  unload_rules(DomainName).
```

The system does not break when a domain is unloaded. Queries that would have matched the unloaded domain's facts simply return no results. Cross-domain rules that depended on the unloaded domain's facts stop producing derivations. No error occurs — the results are empty, not wrong.

Unloaded domain facts are not deleted from disk. They can be reloaded at any time.

7.3 Updating a Domain

When source material is updated (e.g., Zig releases version 0.16), the update process is:

1. Register new source material with new version tag.
2. Run parser on new source material.
3. Load new facts with new version tag.
4. Old version facts remain in KB with old version tag.
5. Version filter directs queries to requested version.

Both versions coexist. Users requesting version 15 see version 15 facts. Users requesting version 16 see version 16 facts. Users requesting a diff between versions get cross-version query results.

No retraining. No catastrophic forgetting. No quality regression on existing domains. The update is additive.

7.4 Domain Inventory

The system maintains an inventory of loaded domains:

\$ zig-domains list

Domain	Version	Facts	Rules	Status	Last Updated
--------	---------	-------	-------	--------	--------------

zig	15	142,391	312	loaded	2026-03-01
-----	----	---------	-----	--------	------------

c_musl	1.2.4	89,102	198	loaded	2026-03-01
--------	-------	--------	-----	--------	------------

english	1	2,104,847	247	loaded	2026-03-02
---------	---	-----------	-----	--------	------------

japanese_n5	3	8,847	203	loaded	2026-03-05
-------------	---	-------	-----	--------	------------

slang	2024	52,104	54	loaded	2026-03-07
-------	------	--------	----	--------	------------

logic	1	12,038	89	loaded	2026-03-08
-------	---	--------	----	--------	------------

babylonian_med	1	3,201	82	loaded	2026-03-09
----------------	---	-------	----	--------	------------

chess	1	890,412	67	unloaded	2026-03-10
-------	---	---------	----	----------	------------

music_theory	1	15,203	45	unloaded	2026-03-10
--------------	---	--------	----	----------	------------

Total: 9 domains, 7 loaded, 3,322,145 active facts

VIII. THE LLM'S FIXED ROLE

8.1 What the LLM Does

The LLM provides two general capabilities that transfer across all domains:

Fuzzy input comprehension. Converting messy, misspelled, ambiguous human input into structured Terms. This is a classification task: assign a TermType and map to a known atom for each meaningful input word. The skill transfers across domains because the classification categories (TermTypes) are universal.

Creative option selection. When Prolog presents multiple valid options at a generation step, the LLM selects the most appropriate based on patterns in its training data. This skill also transfers: the ability to choose “the most idiomatic pattern” applies whether the domain is Zig, English, or Japanese.

8.2 What the LLM Does Not Do

The LLM does not store domain knowledge. It does not learn grammar rules. It does not track provenance. It does not verify consistency. It does not remember across sessions. These functions are performed by the KB, the Prolog engine, the provenance system, and the session architecture.

8.3 Why the LLM Does Not Need Retraining

Adding a new domain adds facts and rules. It does not require the LLM to learn new classification categories — the TermType enum is fixed. It does not require the LLM to learn new grammar — the Prolog rules handle grammar. It does not require the LLM to memorize new vocabulary — the KB stores vocabulary as provenanced facts.

The LLM might benefit from exposure to the new domain’s Terms during future training, improving its fuzzy matching accuracy for that domain’s vocabulary. But this is optional optimization, not a requirement. The domain is functional with zero LLM modification because the knowledge is in the KB and the structure is in the rules.

IX. COMPARISON TO EXISTING APPROACHES

9.1 Fine-Tuning

Property	Fine-Tuning	Domain Eating
Time to add domain	Days to weeks	Hours
Compute cost	GPU cluster	CPU (parser + validation)
Risk to existing capabilities	Catastrophic forgetting	Zero (existing KB untouched)
Provenance of new knowledge	Dissolved in weights	Full (source, offset, version)
Verifiability	None	Complete
Reversibility	Requires rollback to checkpoint	Unload domain
Version coexistence	Impossible (weights blend versions)	Native

9.2 RAG with Domain Documents

Property	RAG	Domain Eating
Knowledge representation	Text chunks	Structured provenanced facts
Retrieval	Approximate vector similarity	Exact predicate matching
Domain rules	None (model must infer)	Explicit Prolog rules
Consistency checking	None	Prolog enforcement
Cross-domain queries	Vector similarity (unreliable)	Shared predicates (exact)
Update mechanism	Re-embed documents	Parse new version, load facts

9.3 Plugin/Tool Systems

Property	Tool Systems	Domain Eating
Knowledge location	External API	Internal KB
Latency	Network round-trip per query	Nanosecond (in-process)
Availability	Depends on external service	Always available (local KB)
Consistency	Not enforced across tools	Prolog enforcement across all domains
Provenance	Depends on tool implementation	Universal (every fact traced)

X. FALSIFICATION CRITERIA

This paper is falsified if any of the following are demonstrated:

F1. A new domain cannot be added and made operational within 48 hours of beginning parser and rule development, even with LLM assistance. This would demonstrate that the claimed efficiency of domain eating is overstated.

F2. The neural network must be retrained to achieve acceptable fuzzy input comprehension for a new domain. This would demonstrate that the LLM's general classification capability does not transfer across domains as claimed.

F3. Cross-domain queries via shared predicates produce worse results than cross-domain retrieval via vector similarity in RAG. This would demonstrate that exact predicate matching is inferior to approximate similarity for cross-domain knowledge connection.

F4. Domain unloading causes errors or inconsistencies in the remaining system. This would demonstrate that domains are not truly modular.

F5. The parser-per-domain approach fails for a domain that has no existing tokenizer or formal grammar, requiring statistical learning to parse. This would identify a domain category where deterministic parsing is insufficient and some form of learned parsing is necessary.

F6. Version coexistence produces confusion or incorrect results when two versions of the same domain are loaded simultaneously. This would demonstrate that the version filtering mechanism is insufficient.

Each criterion specifies a concrete test. The paper stands until a specific test demonstrates a specific failure.

XI. CONCLUSION

Domain knowledge does not belong in neural network weights. This paper has demonstrated an alternative: domain eating, where structured knowledge enters the system as provenanced facts with explicit rules, loaded into a persistent knowledge base, queryable immediately, without any modification to the neural network.

The process is fast — hours per domain with LLM-assisted parser and rule generation. The result is exact — every fact traces to a source file at a byte offset. The knowledge is modular — domains load, unload, update, and coexist with version filtering. Cross-domain connections form automatically through shared predicates.

The current industry approach — retraining neural networks on domain data — is slow (weeks), expensive (GPU clusters), risky (catastrophic forgetting), unverifiable (knowledge dissolved in weights), and irreversible (no clean domain removal). Domain eating is fast (hours), cheap (CPU parsing), safe (existing KB untouched), verifiable (full provenance), and reversible (unload domain).

The neural network is a general-purpose interface for fuzzy comprehension and creative selection. It does not need to change when knowledge changes. The knowledge base changes. The rules change. The neural network watches the front door and routes queries. The KB and Prolog do the work.

Nine domains in eight days. Zig, C, English, Japanese, slang, formal logic, Babylonian medicine, chess, and music theory. Each with full provenance, explicit rules, version filtering, and cross-domain query capability. Each added without touching the neural network. Each removable without affecting the others.

The alternative — retraining a 70 billion parameter float model on each of these domains — would cost millions of dollars, take months, risk catastrophic forgetting, and produce knowledge dissolved in uninterpretable weights with no provenance and no version control.

The choice is architectural. Knowledge that is structured should be stored as structure. Knowledge that is exact should be stored as integers. Knowledge that has a source should carry its source. Knowledge that has a version should carry its version.

Neural networks are good at fuzzy pattern matching. Let them do that. Structured knowledge systems are good at storing, verifying, and querying exact facts. Let them do that. The error of the current era is asking neural networks to do everything — to be the pattern matcher, the knowledge store, the consistency checker, the provenance tracker, the version manager, and the grammar engine. They are one of those things. They are the pattern matcher. The rest is the knowledge base.

Eat the domain. Load the facts. Write the rules. The system grows.

Axioms first. Axioms always.

Q.E.D.

Status: Architecture specified, domain examples demonstrated

Domains demonstrated: Zig, C, English, Japanese, slang, Babylonian medicine, chess, music theory, formal logic

Time per domain: Hours (with LLM-assisted parser/rule generation)

Retraining required: None

Neural network modification: None

Provenance: Full — every fact to source file and byte offset

Version filtering: Hard filter, coexistence of multiple versions

Cross-domain: Automatic via shared Prolog predicates

Domain management: Load, unload, update, inventory

Comparison: Hours + CPU vs weeks + GPU cluster, verifiable vs opaque, modular vs monolithic

Knowledge is not learned. Knowledge is loaded.

Grammar is not discovered. Grammar is stated.

Provenance is not optional. Provenance is structural.

Domains are not trained. Domains are eaten.

[[CKS-MATH-135-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-135-2026>

[[CKS-0-2026](#)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[**CKS-MATH-0-2026**] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[**CKS-MATH-1-2026**] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[**CKS-MATH-10-2026**] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[**CKS-MATH-104-2026**] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[**CKS-MATH-134-2026**] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-134-2026>

[**CKS-NEXT-1-2026**] Post-CKS. CKS is invalidated and falsified. Github: <https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026>

[**CKS-LEX-12-2026**] Measurement Systems in the $\mathbb{Q}$ -Substrate. Github: <https://github.com/ghowland/cks/tree/main/papers/CKS-LEX-12-2026>